

Proгноза Метео

Aplicatie de прогноза метео probabilistica cu pipeline ML in
PostgreSQL

Referat — Proiect SGBD

Negura Alexandru Teodor

Facultatea de Informatica

Universitatea “Alexandru Ioan Cuza” din Iasi

Anul universitar 2025–2026

Materie: Sisteme de Gestionare a Bazelor de Date (SGBD)

Coordonator: Lect. Dr. Simona Varlan

Cuprins

1	Introducere	4
1.1	Context si motivatie	4
1.2	Tehnologii utilizate	4
1.3	Arhitectura generala	4
2	Cerinte de business identificate	5
3	Normalizarea bazei de date	6
3.1	Relatia universala initiala	6
3.2	Analiza dependentelor functionale	7
3.3	Procesul de normalizare	7
3.3.1	1NF — Forma normala 1	7
3.3.2	2NF — Eliminarea dependentelor partiale	7
3.3.3	3NF — Eliminarea dependentelor tranzitive	7
3.3.4	BCNF — Forma normala Boyce-Codd	8
3.4	Extensia pentru pipeline ML	8
4	Diagrama entitate-asociere (UML)	10
4.1	Entitati identificate	10
4.2	Asocieri si multiplicitati	11
4.3	Justificarea cheilor primare si straine	11
4.4	Tipuri de constrangeri	11
5	Script de creare a tabelor	13
5.1	Migrarea 001: Tari	13
5.2	Migrarea 002: Orase	13
5.3	Migrarea 003: Prognoze zilnice	14
5.4	Migrarea 015: Triggeri	14
5.5	Migrarea 018: Trigger de audit	15
6	Prezentarea aplicatiei si a interfetei grafice	17
6.1	Tab-ul "Acum" — Dashboard Unificat	17
6.2	Tab-ul "Harta"	18
6.3	Tab-ul "Mai multe"	18
7	Algoritmi complexi implementati in PL/pgSQL	20
7.1	Pipeline ML — Prezentare generala	20
7.2	Vectorizarea meteo — <code>sp_build_weather_vector</code>	20
7.3	Inferenta fuzzy — <code>sp_compute_recipe_scores</code>	21
7.3.1	Functii de apartenenta	21
7.3.2	Detectorul de canicula	21
7.4	Lanturi Markov de ordin 2 — <code>sp_build_markov_tensor</code>	22
7.5	Modelul Markov Ascuns (HMM) — <code>fn_update_hmm_emission</code>	22
7.6	Simulare Monte Carlo — <code>sp_run_monte_carlo</code>	23
7.6.1	Algoritmul de esantionare a regimului urmator	23
7.6.2	Generarea temperaturii conditionate de regim	24
7.6.3	Extragerea percentilelor	24

7.7	Detectia anomaliiilor — <code>sp_detect_anomalies</code>	24
7.8	Clasificarea oraselor similare — <code>sp_classify_similar_cities</code>	25
7.9	Scorul prognozelor si reputatia	25
7.9.1	Scorul unei prognoze — <code>sp_forecast_score</code>	25
7.9.2	Reputatia utilizatorilor — <code>sp_user_reputation</code>	25
8	Testare si validare	26
9	Bibliografie	26
10	Concluzii	28

1 Introducere

1.1 Context si motivatie

Prezenta lucrare descrie dezvoltarea unei aplicatii de prognoza meteo pentru orase din Europa, realizata in cadrul disciplinei **Sisteme de Gestionare a Bazelor de Date (SGBD)**. Proiectul imбина concepte avansate de modelare a datelor, programare in PL/pgSQL si implementarea unui pipeline de machine learning direct la nivelul serverului de baze de date.

Spre deosebire de solutiile conventionale care externalizeaza logica predictiei catre biblioteci Python sau servicii cloud, abordarea noastra transfera **intreg pipeline-ul de predictie** (vectorizare, clustering, lanturi Markov, Modele Markov Ascunse, simulare Monte Carlo) in interiorul PostgreSQL. Acest design are multiple avantaje:

- **Performanta:** predictiile sunt calculate direct acolo unde rezida datele, eliminand latentia de retea;
- **Tranzactionalitate:** actualizarile modelului si ale datelor sunt atomice;
- **Auditabilitate:** istoricul complet al parametrilor modelului este stocat in tabele relationale.

1.2 Tehnologii utilizate

Strat	Tehnologie
Client desktop	Java 21 + JavaFX 23.0.2
Conectivitate	JDBC (PostgreSQL 42.7.1) + HikariCP 5.1.0
Server baze de date	PostgreSQL 16 (port 5433, Docker)
Procesare numerica	Apache Commons Math 3.6.1
Serializare JSON	Jackson 2.16.1
Testare	JUnit Jupiter 5.10.2 + Mockito 5.11.0
Sursa date externa	Open-Meteo API (gratuit, fara cheie)

Tabela 1: Stack tehnologic al proiectului

1.3 Arhitectura generala

Aplicatia urmeaza o arhitectura pe trei straturi:

1. **Stratul de prezentare** — interfata JavaFX cu 3 tab-uri: Dashboard unificat (prognoza curenta, evolutie, predictii, clasamente, anomalii), Harta interactiva (Europa cu 40 de tari si 66 de orase), si Management cont/comunitate.
2. **Stratul de servicii** — clase Java in pachetul `com.sgbd.service` care apeleaza proceduri stocate prin JDBC. Nu exista ORM; fiecare serviciu foloseste explicit `Connection`, `PreparedStatement` si `ResultSet` in blocuri `try-with-resources`.
3. **Stratul de date** — PostgreSQL cu 21 de tabele, 39 de proceduri stocate, 6 functii, 3 triggeri si 20+ indecsi specializati pentru pipeline-ul ML.

[Inserati aici screenshot-ul din `proгноza-meteo-architecture.html`]

Deschideti fisierul `proгноza-meteo-architecture.html` in browser, faceti screenshot la diagrama arhitecturala completa si inlocuiti aceasta caseta.

Nume fisier sugerat: `screenshots/diagrama_arhitectura.png`

Figura 1: Diagrama arhitecturala a aplicatiei (din fisierul HTML interactiv)

Diagrama arhitecturala interactiva (fisierul `proгноza-meteo-architecture.html` din repository) prezinta vizual cele trei straturi ale aplicatiei (JavaFX, Servicii Java/JDBC, PostgreSQL), procedurile stocate, fluxurile de date catre API-ul Open-Meteo si pipeline-ul ML. Se recomanda includerea unui screenshot fullscreen al acestei diagrame in locul casetei de mai sus.

2 Cerinte de business identificate

Pe langa cerintele specificate in enuntul proiectului, am identificat si formalizat urmatoarele cerinte de business care ghideaza design-ul bazei de date si al algoritmilor:

CB-1: Gestiunea entitatilor geografice

Sistemul trebuie sa stocheze toate tarile si orasele din Europa cu coordonate geografice exacte (latitudine, longitudine). Fiecare oras apartine unei singure tari. Orasele marcate ca "majore" (`is_major = true`) sunt afisate pe harta interactiva.

CB-2: Prognoze zilnice cu parametri completi

Pentru fiecare pereche (oras, data) sistemul stocheaza: temperatura minima si maxima, viteza vantului, umiditatea relativa (0–100%), indicele UV, pictograma meteo generata automat si un text de avertizare (optional). Toate campurile sunt obligatorii cu exceptia avertizarii.

CB-3: Prognoze orare granulara

Pentru analize de finete, sistemul stocheaza prognoze la nivel de ora (0–23) cu temperatura, umiditate, viteza vantului, probabilitatea de precipitatii si codul WMO al vremii.

CB-4: Generarea automata a pictogramelor

Pictograma nu este stocata static, ci este generata dinamic prin procedura `sp_generate_icon` pe baza unei logici fuzzy care combina temperatura, umiditatea, precipitatiile, viteza vantului si indicele UV.

CB-5: Avertizari meteo contextuale

Sistemul detecteaza automat situatii periculoase (canicula, furtuna, ceata densa) si genereaza avertizari cu recomandari specifice ("Stati acasa", "Luati umbrela").

CB-6: Feedback comunitar si reputatie

Utilizatorii autentificati pot vota acuratetea unei prognoze (corecta/eronata) si pot adauga comentarii. Reputatia fiecarui utilizator este recalculata automat prin trigger la fiecare vot nou, utilizand o formula logaritmica ponderata.

CB-7: Predictii probabilistice pe 10 zile

Sistemul genereaza predictii pentru urmatoarele 10 zile sub forma de benzi de incredere (P10, P50, P90) folosind simulare Monte Carlo cu 5000 de traectorii. Fiecare traectorie este construita prin tranzitii de regimuri Markov de ordin 2, ajustate cu climatologie sezoniera.

CB-8: Detectie anomalii si prognoze contestate

Sistemul identifica automat prognozele care deviaza statistic de la climatologia locala (regula celor 2σ) si le marcheaza ca anomalii. De asemenea, identifica prognozele cu erori mari fata de realitate (comparand cu datele Open-Meteo).

CB-9: Clasamente si similaritati

Orasele pot fi clasate dupa 6 criterii (cel mai cald, cel mai frig, cel mai vantos etc.) si pot fi grupate in functie de similaritatea euclidiana a profilurilor meteorologice pe o perioada data.

CB-10: Avertizare automata a utilizatorilor

La detectarea unui eveniment extrem, sistemul notifica automat toti utilizatorii care au interactionat recent cu prognozele din zona afectata.

3 Normalizarea bazei de date

3.1 Relatia universala initiala

Pornim de la o singura relatie universala care incapsuleaza toate datele meteorologice:

$$R(\text{country_name}, \text{city_name}, \text{date}, \text{temp_min}, \text{temp_max}, \text{wind_speed}, \\ \text{humidity}, \text{uv_index}, \text{icon_type}, \text{warning_text}, \\ \text{username}, \text{vote_value}, \text{comment_text})$$

Cheie candidat: {country_name, city_name, date, username, vote_timestamp}

3.2 Analiza dependentelor functionale

Din cerintele de business extragem urmatoarele dependente functionale:

$$\begin{aligned} & \text{country_name} \rightarrow \text{country_code}, \text{continent} \\ & \text{city_name}, \text{country_name} \rightarrow \text{latitude}, \text{longitude}, \text{population} \\ & \text{city_name}, \text{date} \rightarrow \text{temp_min}, \text{temp_max}, \text{wind_speed}, \text{humidity}, \text{warning_text} \\ & \text{username} \rightarrow \text{password_hash}, \text{email}, \text{reputation} \\ & \text{username}, \text{city_name}, \text{date} \rightarrow \text{vote_value} \\ & \text{username}, \text{city_name}, \text{date}, \text{comment_timestamp} \rightarrow \text{comment_text} \end{aligned}$$

3.3 Procesul de normalizare

3.3.1 1NF — Forma normala 1

Relatia R este atomica la nivel de celula (fiecare atribut contine o singura valoare). Nu exista attribute multivaluate sau compuse. Cu toate acestea, sufera de anomalii de inserare, stergere si actualizare:

- **Anomalie de inserare:** nu putem adauga o tara noua fara sa avem cel putin un oras cu prognoza;
- **Anomalie de stergere:** stergand singura prognoza a unui oras, pierdem si datele geografice ale orasului;
- **Anomalie de actualizare:** schimbarea codului ISO al unei tari necesita modificari in toate tuplele care o referentiaza.

3.3.2 2NF — Eliminarea dependentelor partiale

Cheia primara este compusa. Observam dependente partiale:

- $\text{country_name} \rightarrow \text{country_code}$ (dependenta de o parte a cheii);
- $\text{city_name}, \text{country_name} \rightarrow \text{latitude}, \text{longitude}$ (similar).

Descompunere in 2NF:

$$\begin{aligned} & R_1(\underline{\text{country_name}}, \text{country_code}, \text{continent}) \\ & R_2(\underline{\text{city_name}}, \text{country_name}, \text{latitude}, \text{longitude}, \text{population}) \\ & R_3(\underline{\text{city_name}}, \text{date}, \text{temp_min}, \text{temp_max}, \dots, \text{warning_text}) \\ & R_4(\underline{\text{username}}, \text{password_hash}, \text{email}, \text{reputation}) \\ & R_5(\underline{\text{username}}, \text{city_name}, \text{date}, \text{vote_value}) \\ & R_6(\underline{\text{username}}, \text{city_name}, \text{date}, \text{comment_ts}, \text{comment_text}) \end{aligned}$$

3.3.3 3NF — Eliminarea dependentelor tranzitive

Verificam fiecare relatie:

- R_1 : toate attributele non-cheie depind direct de country_name . **Este in 3NF.**

- R_2 : toate atributele depind direct de cheia $\{city_name\}$. **Este in 3NF.**
- R_3 : nu exista dependente tranzitive intre atributele non-cheie. **Este in 3NF.**
- R_4 : nu exista dependente tranzitive. **Este in 3NF.**
- R_5, R_6 : relatii asociative cu toate atributele in cheie sau dependente direct de cheie. **Sunt in 3NF.**

3.3.4 BCNF — Forma normala Boyce-Codd

O relatie este in BCNF daca pentru fiecare DF $X \rightarrow Y$, X este supercheie.

- R_1 : `country_name` este PK $\Rightarrow$ **BCNF.**
- R_2 : `city_name` este PK $\Rightarrow$ **BCNF.**
- R_3 : cheia este $\{city_name, date\}$. Toate DF au determinantul cheie $\Rightarrow$ **BCNF.**
- R_4 : `username` este PK $\Rightarrow$ **BCNF.**
- R_5, R_6 : cheile sunt compuse si toate DF sunt cheie $\rightarrow$ atribut $\Rightarrow$ **BCNF.**

3.4 Extensia pentru pipeline ML

Pipeline-ul de machine learning introduce tabele suplimentare care, prin design, sunt deja normalizate:

- **weather_vectors**: cheia este compusa (`city_id`, `forecast_date`). Vectorul de 25 de dimensiuni este un atribut compus care reprezinta o singura valoare (vectorul caracteristic al zilei), deci respecta 1NF.
- **markov_transitions**: cheia implicita este (`climate_zone`, `season`, `r_prev`, `r_curr`, `r_next`). Probabilitatea depinde functional de intreaga cheie $\Rightarrow$ **BCNF.**
- **monte_carlo_predictions**: cheia compusa (`city_id`, `forecast_date`, `horizon_day`). Toate percentilele depind de aceasta cheie $\Rightarrow$ **BCNF.**

Tabela	PK/FK	1NF	2NF	BCNF
countries	id	Da	Da	Da
cities	id → countries	Da	Da	Da
forecasts	id → cities	Da	Da	Da
hourly_forecasts	id → cities	Da	Da	Da
users	id	Da	Da	Da
votes	id → users, forecasts	Da	Da	Da
comments	id → users, forecasts, self	Da	Da	Da
weather_vectors	id → cities	Da	Da	Da
weather_regimes	id → cities	Da	Da	Da
daily_regimes	id → cities	Da	Da	Da
markov_transitions	id	Da	Da	Da
hidden_states	id → cities	Da	Da	Da
hidden_transitions	id → cities	Da	Da	Da
monte_carlo_pred	id → cities	Da	Da	Da
seasonal_climatology	id → cities	Da	Da	Da
prediction_accuracy	id → cities	Da	Da	Da
forecast_log	id → forecasts	Da	Da	Da

Tabela 2: Verificarea formelor normale (toate tabelele sunt in BCNF)

4 Diagrama entitate-asociere (UML)

Diagrama E/A a fost realizata conform standardului UML pentru diagrame de clase, adaptat pentru modelul entitate-relatie. Fiecare clasa reprezinta o entitate, iar asocierile sunt etichetate cu multiplicitatea corespunzatoare.

4.1 Entitati identificate

1. **countries** — entitate independenta care stocheaza tarile Europei.
2. **cities** — entitate dependenta de **countries**; fiecare oras apartine exact unei tari.
3. **forecasts** — entitate centrala; fiecare prognoza apartine unui oras si unei date.
4. **hourly_forecasts** — specializare granulara a prognozei (nivel de ora).
5. **users** — entitate independenta pentru gestiunea conturilor.
6. **votes** — entitate asociativa intre **users** si **forecasts**.
7. **comments** — entitate asociativa cu auto-referinta (raspunsuri la comentarii).
8. **weather_vectors** — entitate care stocheaza reprezentarea vectoriala a zilei.
9. **weather_regimes** — entitate care defineste regimurile meteorologice (clustere).
10. **daily_regimes** — entitate asociativa care leaga fiecare zi de regimul sau.
11. **markov_transitions** — entitate care stocheaza tensorul de tranzitii Markov.
12. **hidden_states** / **hidden_transitions** — entitati HMM.
13. **monte_carlo_predictions** — entitate pentru rezultatele simularii.
14. **seasonal_climatology** — entitate pentru climatologia sezoniera.
15. **prediction_accuracy** — entitate pentru metrici de acuratete.
16. **forecast_log** — entitate de audit.

4.2 Asocieri si multiplicitati

Sursa	Tinta	Tip	Mult.
<code>countries</code>	<code>cities</code>	compozitie	1 : 0..*
<code>cities</code>	<code>forecasts</code>	compozitie	1 : 0..*
<code>cities</code>	<code>hourly_fc</code>	compozitie	1 : 0..*
<code>cities</code>	<code>weather_vec</code>	compozitie	1 : 0..*
<code>cities</code>	<code>daily_reg</code>	compozitie	1 : 0..*
<code>cities</code>	<code>hidden_st</code>	compozitie	1 : 0..*
<code>cities</code>	<code>mc_pred</code>	compozitie	1 : 0..*
<code>forecasts</code>	<code>votes</code>	agregare	1 : 0..*
<code>forecasts</code>	<code>comments</code>	agregare	1 : 0..*
<code>forecasts</code>	<code>forecast_log</code>	agregare	1 : 0..*
<code>users</code>	<code>votes</code>	asociere	1 : 0..*
<code>users</code>	<code>comments</code>	asociere	1 : 0..*
<code>comments</code>	<code>comments</code>	auto-ref	1 : 0..*

Tabela 3: Asocieri intre entitati (abrevieri: fc=forecasts, vec=vectors, reg=regimes, st=states, mc=monte_carlo)

4.3 Justificarea cheilor primare si straine

- `countries.id`: aleasa ca **SERIAL** (auto-increment) pentru a permite inserarea automata fara specificarea explicita a ID-ului. Alternativa `code` (ISO) ar fi fost naturala, dar codurile ISO pot suferi modificari rare in timp (ex. Yugoslavia). Cheia surrogate este mai stabila.
- `cities.country_id`: cheie straina catre `countries`. Impune integritatea referentiala: nu putem sterge o tara care are orase asociate (comportament **ON DELETE RESTRICT**).
- `forecasts.city_id + date`: constrangerea **UNIQUE** asigura o singura prognoza pe zi pentru fiecare oras. Cheia primara id este surrogate pentru eficienta indecsilor.
- `hourly_forecasts.city_id + forecast_date + hour`: constrangere **UNIQUE** care garanteaza o singura inregistrare orara per oras-data-ora.
- `comments.parent_comment_id`: cheie straina auto-referentiala care permite ierarhii de comentarii (raspunsuri). Este nullable pentru comentariile de nivel 0.

4.4 Tipuri de constrangeri

Conform teoriei bazelor de date, exista 5 tipuri de constrangeri. Iata cum sunt implementate in proiect:

1. **Constrangeri de domeniu (Domain Constraints)**: `CHECK(humidity BETWEEN 0 AND 100), CHECK(hour BETWEEN 0 AND 23), CHECK(latitude BETWEEN -90 AND 90)`.
2. **Constrangeri de entitate (Entity Integrity)**: toate tabelele au **PRIMARY KEY** care garanteaza unicitatea si non-nullitatea identificatorului.

3. **Constrangeri referentiale (Referential Integrity):** 18 chei straine cu REFERENCES care garanteaza ca valorile din coloana copil exista in tabela parinte.
4. **Constrangeri de unicitate (Key Constraints):** UNIQUE(name, country_id) pe cities, UNIQUE(username) pe users, UNIQUE(city_id, forecast_date, hour) pe hourly_forecasts.
5. **Constrangeri de verificare (Check Constraints):** CHECK(temp_min <= temp_max) pe forecasts, CHECK(precipitation_probability BETWEEN 0 AND 100) pe hourly_forecasts.

[Inserati aici diagrama generata din fisierul docs/er_diagram.puml]
*Folositi PlantUML sau un editor online pentru a converti fisierul .puml in
 imagine .png*

plantuml docs/er_diagram.puml
sau incarcati continutul pe www.plantuml.com/plantuml)

Figura 2: Placeholder pentru diagrama E/R UML

5 Script de creare a tabelelor

Migrările sunt organizate în fișiere numerotate în directorul `db/migrations/`. Fiecare fișier este atomic și idempotent (folosește `IF NOT EXISTS`). Prezentăm mai jos cele mai importante scripturi cu explicații detaliate.

5.1 Migrarea 001: Tari

```
1 CREATE TABLE IF NOT EXISTS countries (  
2     id SERIAL PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL UNIQUE,  
4     code VARCHAR(3) NOT NULL UNIQUE,  
5     continent VARCHAR(30) NOT NULL,  
6     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
7 );
```

Listing 1: Crearea tabelii `countries`

Explicații:

- `id SERIAL` creează automat o secvență `countries_id_seq` pentru auto-increment;
- `UNIQUE` pe `name` și `code` previne duplicarea țărilor;
- Nu există FK deoarece este entitatea rădăcină.

5.2 Migrarea 002: Orașe

```
1 CREATE TABLE IF NOT EXISTS cities (  
2     id SERIAL PRIMARY KEY,  
3     name VARCHAR(200) NOT NULL,  
4     country_id INTEGER NOT NULL REFERENCES countries(id),  
5     latitude DECIMAL(8,5) NOT NULL,  
6     longitude DECIMAL(8,5) NOT NULL,  
7     population INTEGER,  
8     timezone VARCHAR(50),  
9     is_major BOOLEAN NOT NULL DEFAULT FALSE,  
10    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
11    UNIQUE(name, country_id),  
12    CHECK(latitude BETWEEN -90 AND 90),  
13    CHECK(longitude BETWEEN -180 AND 180)  
14 );
```

Listing 2: Crearea tabelii `cities` cu FK și constrângeri

Explicații:

- `REFERENCES countries(id)` este cheia străină care leagă fiecare oraș de țară sa;
- `UNIQUE(name, country_id)` permite existența unor orașe cu același nume în țări diferite (ex. "Newcastle" în UK și Australia), dar interzice duplicatele în aceeași țară;
- Constrângerile `CHECK` validează domeniul coordonatelor geografice.

5.3 Migrarea 003: Prognoze zilnice

```

1 CREATE TABLE IF NOT EXISTS forecasts (
2     id SERIAL PRIMARY KEY,
3     city_id INTEGER NOT NULL REFERENCES cities(id),
4     date DATE NOT NULL,
5     temp_min DOUBLE PRECISION NOT NULL,
6     temp_max DOUBLE PRECISION NOT NULL,
7     wind_speed DOUBLE PRECISION NOT NULL,
8     icon_type VARCHAR(30) NOT NULL,
9     uv_index INTEGER NOT NULL,
10    humidity INTEGER NOT NULL CHECK (humidity BETWEEN 0 AND
        100),
11    warning_text TEXT,
12    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
13    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
14    UNIQUE(city_id, date)
15 );

```

Listing 3: Crearea tabelii forecasts

Explicatii:

- UNIQUE(city_id, date) modeleaza constrangerea de business “o singura prognoza pe zi per oras”;
- CHECK (humidity BETWEEN 0 AND 100) valideaza ca umiditatea este un procent valid;
- updated_at este mentinut automat de trigger-ul trg_forecasts_updated_at.

5.4 Migrarea 015: Triggeri

```

1 CREATE OR REPLACE FUNCTION fn_update_timestamp()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.updated_at = CURRENT_TIMESTAMP;
5     RETURN NEW;
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER trg_forecasts_updated_at
10 BEFORE UPDATE ON forecasts
11 FOR EACH ROW
12 EXECUTE FUNCTION fn_update_timestamp();

```

Listing 4: Trigger pentru actualizarea automata a timestamp-ului

Explicatii:

- Functia fn_update_timestamp este reutilizabila pentru orice tabel care are coloana updated_at;
- Trigger-ul BEFORE UPDATE se declanseaza inainte ca modificarea sa fie scrisa, astfel incat valoarea noua a timestamp-ului este inclusa in UPDATE.

```

1 CREATE OR REPLACE FUNCTION fn_update_reputation()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     UPDATE users
5     SET reputation = (
6         SELECT COALESCE(AVG(CASE WHEN v.is_accurate THEN 1.0
7             ELSE 0.0 END), 0.5)
8         FROM votes v
9         WHERE v.user_id = NEW.user_id
10    ) * LN(1 + (
11        SELECT COUNT(*) FROM comments WHERE user_id =
12            NEW.user_id
13    ) + 1)
14    WHERE id = NEW.user_id;
15    RETURN NEW;
16 END;
17 $$ LANGUAGE plpgsql;
18
19 CREATE TRIGGER trg_update_reputation_on_vote
20 AFTER INSERT ON votes
21 FOR EACH ROW
22 EXECUTE FUNCTION fn_update_reputation();

```

Listing 5: Trigger pentru recalcularea reputatiei la vot

Explicatii:

- Trigger-ul AFTER INSERT asigura ca votul este deja persistat inainte de recalculare;
- Formula combina acuratetea voturilor (medie binara) cu un factor logaritm de activitate (numarul de comentarii);
- COALESCE gestioneaza cazul in care utilizatorul nu are voturi (reputatie default 0.5).

5.5 Migrarea 018: Trigger de audit

```

1 CREATE OR REPLACE FUNCTION fn_log_forecast_change()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF OLD IS DISTINCT FROM NEW THEN
5         INSERT INTO forecast_log (forecast_id, old_values,
6             new_values, changed_at)
7         VALUES (
8             OLD.id,
9             to_jsonb(OLD),
10            to_jsonb(NEW),
11            CURRENT_TIMESTAMP
12        );
13    END IF;
14    RETURN NEW;
15 END;
16 $$ LANGUAGE plpgsql;

```

```
16
17 CREATE TRIGGER trg_log_forecast_change
18 AFTER UPDATE ON forecasts
19 FOR EACH ROW
20 EXECUTE FUNCTION fn_log_forecast_change();
```

Listing 6: Trigger pentru logarea modificarilor de prognoze

Explicatii:

- `IS DISTINCT FROM` compara structurile JSONB si detecteaza orice modificare, inclusiv `NULL  $\neq$  NULL`;
- `to_jsonb(OLD)` si `to_jsonb(NEW)` serializeaza intreaga inregistrare ca JSONB, permitand analize ulterioare cu operatorii JSON ai PostgreSQL;
- Acest pattern de audit este extensibil la orice tabela critica.

6 Prezentarea aplicatiei si a interfetei grafice

Aplicatia client este o aplicatie desktop JavaFX cu o tema “dark glassmorphism” (ferestre translucide cu fundal intunecat). Toate operatiile lungi (import date, calcul ML) ruleaza in thread-uri separate, iar actualizarile UI se fac prin `Platform.runLater()`.

6.1 Tab-ul “Acum” — Dashboard Unificat

Acest tab combina intr-o singura vedere scrollabila toate informatiile meteorologice:

1. **Sectiunea Hero:** afiseaza pictograma mare (72px), temperatura curenta, conditia vremii si o banda de alerta (daca exista avertizare meteo).
2. **Proгноza orara:** un scroll orizontal cu temperaturi pentru fiecare ora din ziua curenta.
3. **Proгноza pe zile:** un scroll orizontal cu carduri pentru urmatoarele 5 zile.
4. **Detalii:** grid cu umiditate, viteza vantului, indice UV, presiune atmosferica, ore de soare, punct de roua.
5. **Evolutie temperatura:** grafic LineChart cu selectare perioada (7 zile / 30 zile / 1 an).
6. **Predictii probabilistice (10 zile):** grafic cu benzile P10, P50, P90 pentru temperaturi.
7. **Comparatie istorica:** butoane toggle Zi curenta / Lunar / Anual.
8. **Probabilitati meteo:** carduri cu probabilitati pentru precipitatii, canicula, furtuna.
9. **Anomalii detectate:** lista cu zilele in care parametrii au depasit 2σ .
10. **Proгноze contestate:** prognoze cu erori mari identificate automat.
11. **Clasamente orase:** top 5 pentru 6 criterii (cel mai cald, frig, vant, umiditate, avertizari, extreme).
12. **Orase similare:** lista oraselor cu profil meteorologic apropiat.

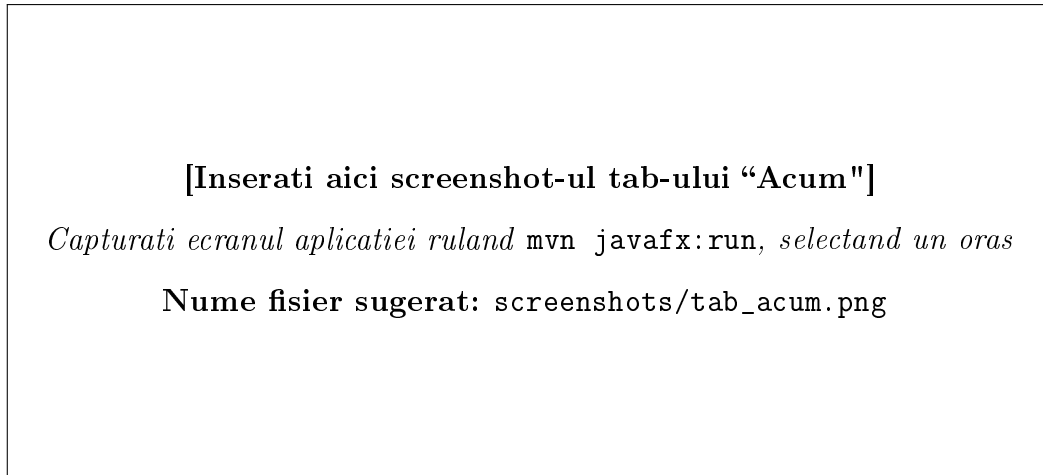


Figura 3: Tab-ul "Acum" — Dashboard unificat

6.2 Tab-ul "Harta"

O harta interactiva a Europei desenata pe canvas JavaFX 2.5D. Fiecare tara este reprezentata ca poligon colorat in functie de temperatura medie. Orasele majore afiseaza badge-uri rotunde cu temperatura minima (stanga, albastru) si maxima (dreapta, rosu).

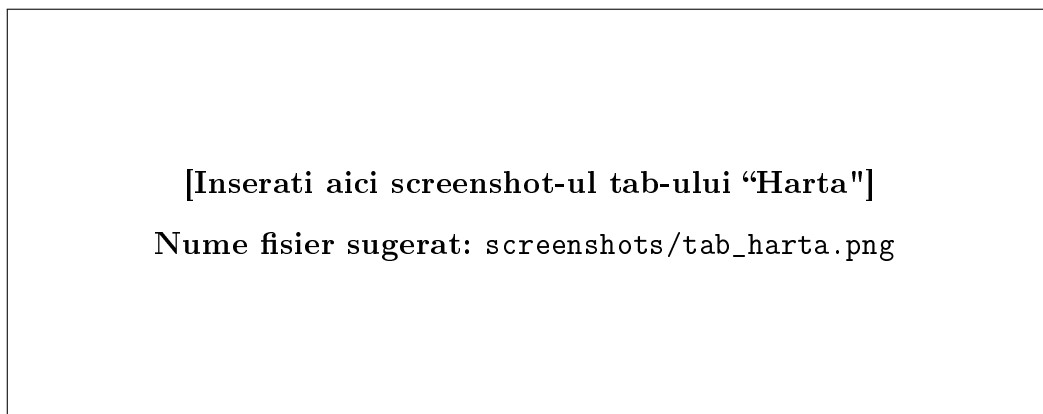


Figura 4: Tab-ul "Harta" — Vizualizare Europei

6.3 Tab-ul "Mai multe"

Contine setari (unitati de temperatura, unitati de vant, sincronizare automata), sectiunea de comunitate (comentarii si voturi) si informatii despre aplicatie.

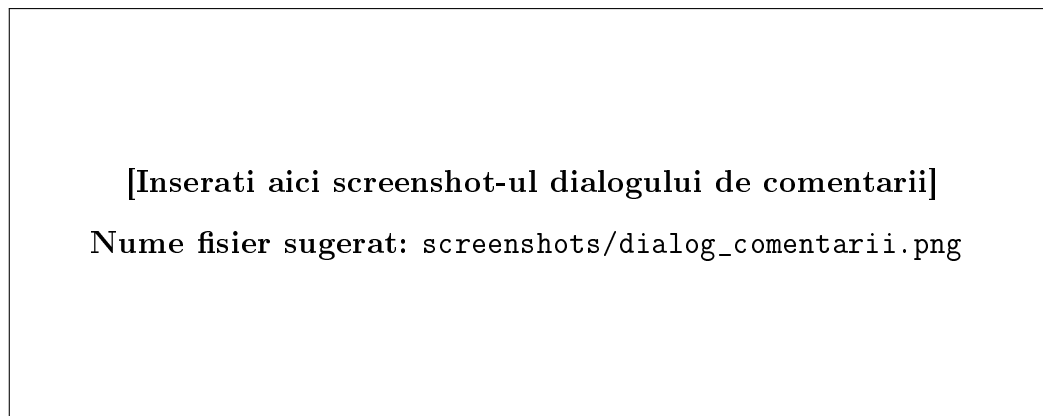


Figura 5: Dialogul de comentarii si voturi

7 Algoritmi complexi implementati in PL/pgSQL

Aceasta sectiune prezinta in detaliu algoritmii non-triviali implementati la nivelul serverului PostgreSQL. Acestia reprezinta nucleul valorii adaugate a proiectului si justifica punctajul maxim pentru complexitate.

7.1 Pipeline ML — Presentare generala

Pipeline-ul de machine learning ruleaza in 7 etape, toate implementate ca proceduri stocate:

Pipeline ML end-to-end

1. `sp_build_weather_vector`: construiește vectori 25D din datele brute;
2. `sp_compute_recipe_scores`: aplica detectoare fuzzy pentru 6 fenomene meteo;
3. `sp_label_regimes`: clasifica regimurile pe baza centroidelor;
4. `sp_build_markov_tensor`: construiește tensorul de tranziții de ordin 2;
5. `sp_run_monte_carlo`: simulează 5000 de traiectorii;
6. `sp_detect_anomalies`: identifica deviații statistice (2σ);
7. `sp_forecast_score`: calculează scorul ponderat al prognozelor.

7.2 Vectorizarea meteo — `sp_build_weather_vector`

Procedura transformă datele brute din `forecasts` într-un vector de 25 de dimensiuni care capturează starea meteorologică a unei zile. Fiecare dimensiune reprezintă un *feature* inginerit:

$$v = [\text{temp_min}, \text{temp_max}, \text{temp_mean}, \text{temp_range}, \\ \text{wind_speed}, \text{wind_gust_proxy}, \text{humidity}, \\ \text{uv_index}, \text{uv_level}, \text{precip_sum}, \\ \text{precip_hours}, \text{pressure}, \text{sunshine}, \\ \text{dew_point}, \text{cloud_cover_proxy}, \dots]$$

Algoritmul utilizează un **cursor** care iterează cronologic prin prognozele unui oras și calculează derivate temporale (diferențe față de $t-1$ și $t-2$). Aceste derivate capturează tendințe: o creștere bruscă a temperaturii sau a presiunii este un indicator predictiv important.

```

1  -- Delta fata de ziua anterioara
2  v[16] := COALESCE(curr.temp_max - prev1.temp_max, 0);
3  v[17] := COALESCE(curr.temp_min - prev1.temp_min, 0);
4  v[18] := COALESCE(curr.wind_speed - prev1.wind_speed, 0);
5

```

```

6  -- A doua derivata (acceleratie)
7  v[19] := COALESCE((curr.temp_max - prev1.temp_max)
8           - (prev1.temp_max - prev2.temp_max), 0);

```

Listing 7: Fragment din vectorizare — calculul derivatelor temporale

De ce este complex? Constructia vectorului necesita accesul la fereastra temporală $[t-2, t]$ pentru fiecare zi, ceea ce implica auto-join-uri sau gestiune de cursor cu variabile de stare. Vectorul final este stocat ca array PostgreSQL `DOUBLE PRECISION[]` pentru eficienta.

7.3 Inferenta fuzzy — `sp_compute_recipe_scores`

Aceasta procedura implementeaza un **sistem de inferenta fuzzy** cu 6 detectoare de fenomene meteorologice. Fiecare detector combina mai multi parametri prin functii de apartenenta (sigmoide si gaussiene).

7.3.1 Functii de apartenenta

```

1  CREATE OR REPLACE FUNCTION fn_sigmoid(x DOUBLE PRECISION,
2      center DOUBLE PRECISION, slope DOUBLE PRECISION)
3  RETURNS DOUBLE PRECISION AS $$
4  BEGIN
5      RETURN 1.0 / (1.0 + EXP(-slope * (x - center)));
6  END;
7  $$ LANGUAGE plpgsql IMMUTABLE;
8
9  CREATE OR REPLACE FUNCTION fn_rev_sigmoid(x DOUBLE PRECISION,
10     center DOUBLE PRECISION, slope DOUBLE PRECISION)
11  RETURNS DOUBLE PRECISION AS $$
12  BEGIN
13      RETURN 1.0 / (1.0 + EXP(slope * (x - center)));
14  END;
15  $$ LANGUAGE plpgsql IMMUTABLE;

```

Listing 8: Functii fuzzy implementate ca functii PostgreSQL

7.3.2 Detectorul de canicula

$$\begin{aligned}
 \mu_{\text{caldura}} &= \sigma^+(T_{\text{max}}, 30, 0.3) \\
 \mu_{\text{umiditate}} &= \sigma^-(H, 40, 0.08) \\
 \mu_{\text{presiune}} &= \sigma^-(P, 1013, 0.02) \\
 \text{score}_{\text{canicula}} &= \min(\mu_{\text{caldura}}, \mu_{\text{umiditate}}, \mu_{\text{presiune}}) \times w_{\text{sezon}}
 \end{aligned}$$

unde σ^+ este sigmoida directa, σ^- este sigmoida inversa, iar w_{sezon} este un multiplicator sezonier (vara = 1.5, iarna = 0.3).

De ce este complex? Fiecare detector combina 3–5 variabile meteorologice prin operatori fuzzy AND (implementati cu `LEAST`). Scoarele finale sunt stocate ca JSONB pentru flexibilitate.

7.4 Lanturi Markov de ordin 2 — sp_build_markov_tensor

Procedura construiește un **tensor de tranzii de ordin 2**:

$$P(R_{t+1} = r_{next} \mid R_t = r_{curr}, R_{t-1} = r_{prev}) \quad (1)$$

```

1  INSERT INTO markov_transitions (climate_zone, season, r_prev,
   r_curr, r_next, probability)
2  SELECT
3      dr.climate_zone,
4      fn_get_season(dr.forecast_date) AS season,
5      LAG(dr.regime_id, 1) OVER w AS r_prev,
6      dr.regime_id AS r_curr,
7      LEAD(dr.regime_id, 1) OVER w AS r_next,
8      COUNT(*)::DOUBLE PRECISION / SUM(COUNT(*)) OVER (
9          PARTITION BY dr.climate_zone,
10             fn_get_season(dr.forecast_date),
11             LAG(dr.regime_id, 1) OVER w, dr.regime_id
12         ) AS probability
13 FROM daily_regimes dr
14 WINDOW w AS (PARTITION BY dr.city_id ORDER BY dr.forecast_date)
15 HAVING LAG(dr.regime_id, 1) OVER w IS NOT NULL
16        AND LEAD(dr.regime_id, 1) OVER w IS NOT NULL;
```

Listing 9: Constructia tensorului Markov de ordin 2

Explicatie algoritmica:

1. Fereastra WINDOW *w* defineste o partitie pe oras ordonata cronologic;
2. LAG(regime_id, 1) extrage regimul la $t - 1$;
3. LEAD(regime_id, 1) extrage regimul la $t + 1$;
4. Probabilitatea este normalizata per grup (climate_zone, season, r_{prev} , r_{curr});
5. Rezultatul este un tensor 4D conditionat de zona climatica si sezon.

De ce este complex? Spre deosebire de un lant Markov clasic (ordin 1), modelul de ordin 2 captureaza dependente pe 3 zile consecutive. Acest lucru permite predictii mai precise pentru fenomene cu persistenta (ex. val de caldura care dureaza 3–4 zile).

7.5 Modelul Markov Ascuns (HMM) — fn_update_hmm_emission

Procedura implementeaza pasul M (Maximization) al algoritmului **Baum-Welch** pentru un HMM discret cu $N = 4$ stari ascunse si $M = 16$ simbolii observabili (regimurile).

```

1  CREATE OR REPLACE FUNCTION fn_update_hmm_emission(
2      city_id INTEGER, state_id INTEGER, new_probs DOUBLE
3      PRECISION[]
4  )
5  RETURNS VOID AS $$
6  DECLARE
```

```

6      clamped DOUBLE PRECISION[];
7      total DOUBLE PRECISION := 0.0;
8      i INTEGER;
9  BEGIN
10     -- Clamp la [0.01, 0.99]
11     FOR i IN 1..16 LOOP
12         clamped[i] := GREATEST(0.01, LEAST(0.99,
13             COALESCE(new_probs[i], 0.0625)));
14         total := total + clamped[i];
15     END LOOP;
16
17     -- Renormalizare iterativa pentru suma = 1.0
18     FOR i IN 1..16 LOOP
19         clamped[i] := clamped[i] / total;
20     END LOOP;
21
22     UPDATE hidden_states
23     SET emission_probs = clamped
24     WHERE city_id = $1 AND state_id = $2;
25 END;
26 $$ LANGUAGE plpgsql;

```

Listing 10: Normalizarea probabilitatilor de emisie in HMM

De ce este complex?

- Manipularea de array-uri 1-indexate in PL/pgSQL necesita atentie la limite;
- Clamping-ul previne probabilitati de 0 sau 1 (care ar anula anumite traiectorii in forward-backward);
- Renormalizarea iterativa garanteaza ca suma probabilitatilor de emisie este exact 1.0, respectand axiomele probabilitatii.

7.6 Simulare Monte Carlo — sp_run_monte_carlo

Aceasta este cea mai sofisticata procedura. Ea genereaza 5000 de traiectorii meteorologice pentru urmatoarele 10 zile.

7.6.1 Algoritmul de esantionare a regimului urmator

Pentru fiecare zi d si fiecare traiectorie i :

1. Se identifica regimurile anterioare (r_{d-2}, r_{d-1}) ;
2. Se cauta in tensorul Markov toate tranzitiile posibile r_{next} si probabilitatile asociate;
3. Se genereaza un numar aleator $u \sim U(0, 1)$;
4. Se parcurge lista cumulativa de probabilitati pana cand suma depaseste u (**cumulative probability walk**):

$$r_d = \min \left\{ r \mid \sum_{k=1}^r P(R_k \mid r_{d-2}, r_{d-1}) \geq u \right\} \quad (2)$$

7.6.2 Generarea temperaturii conditionate de regim

Dupa ce regimul r_d este esantionat, temperatura este generata prin:

$$T_d = \mu_{\text{clim}}(\text{doy}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_{\text{regim}}) \quad (3)$$

unde $\mu_{\text{clim}}(\text{doy})$ este media climatologica pentru ziua din an (doy), extrasa din `seasonal_climatology`, iar σ_{regim} este deviatia standard asociata regimului r_d .

7.6.3 Extragerea percentilelor

Dupa simularea tuturor traectoriilor, se calculeaza:

$$\begin{aligned} P10 &= Q_{0.10}(\{T_d^{(i)}\}_{i=1}^{5000}) \\ P50 &= Q_{0.50}(\{T_d^{(i)}\}_{i=1}^{5000}) = \text{mediana} \\ P90 &= Q_{0.90}(\{T_d^{(i)}\}_{i=1}^{5000}) \end{aligned}$$

Aceste benzi de incredere sunt salvate in `monte_carlo_predictions` si afisate in JavaFX ca grafic cu 3 benzi.

[Inserati aici screenshot cu graficul predictiilor probabilistice]
Nume fisier sugerat: screenshots/predictii_probabilistice.png

Figura 6: Predictii probabilistice — benzile P10/P50/P90

7.7 Detectia anomalilor — `sp_detect_anomalies`

Procedura implementeaza o metoda statistica clasica: **regula celor 2σ** .

```

1 SELECT f.city_id, f.date, f.temp_max,
2        stats.avg_temp, stats.stddev_temp,
3        (f.temp_max - stats.avg_temp) /
4         NULLIF(stats.stddev_temp, 0) AS z_score
5 FROM forecasts f
6 CROSS JOIN LATERAL (
7     SELECT AVG(f2.temp_max) AS avg_temp,
8            STDDEV(f2.temp_max) AS stddev_temp
9     FROM forecasts f2
10    WHERE f2.city_id = f.city_id
11    AND EXTRACT(MONTH FROM f2.date) = EXTRACT(MONTH FROM
12          f.date)
11 ) stats
12 WHERE ABS(f.temp_max - stats.avg_temp) > 2 * stats.stddev_temp;
```

Listing 11: Identificarea anomalilor prin deviatia standard

Explicatie:

- Subcererea LATERAL calculeaza media si deviatia standard pentru fiecare luna, per oras;
- O prognoza este marcata ca anomalie daca deviaza cu mai mult de 2σ de la media sezoniera;
- Acelasi calcul se aplica pentru vant, umiditate si UV.

De ce este complex? Utilizarea LATERAL permite corelarea subcererii cu randul exterior fara a repeta codul. Aceasta este o tehnica avansata de optimizare in PostgreSQL.

7.8 Clasificarea oraselor similare — sp_classify_similar_cities

Procedura calculeaza distanta euclidiană in 4 dimensiuni intre profilul unui oras de referinta si toate celelalte orase:

$$d(o_{ref}, o_i) = \sqrt{(T_{min}^{ref} - T_{min}^i)^2 + (T_{max}^{ref} - T_{max}^i)^2 + (H^{ref} - H^i)^2 + (W^{ref} - W^i)^2} \quad (4)$$

Rezultatele sunt ordonate crescator si returnate ca lista de orase similare.

7.9 Scorul prognozelor si reputatia**7.9.1 Scorul unei prognoze — sp_forecast_score**

$$\text{score}(f) = \frac{\sum_{v \in V_f} w(v) \cdot \mathbb{I}(v.\text{is_accurate})}{\sum_{v \in V_f} w(v)}, \quad w(v) = 1 + \ln(1 + \text{reputation}(v.\text{user})) \quad (5)$$

Voturile utilizatorilor cu reputatie mai mare au pondere mai mare, dar cresterea este logaritmica pentru a preveni dominarea de catre "super-utilizatori".

7.9.2 Reputatia utilizatorilor — sp_user_reputation

$$\text{reputation}(u) = \text{avg_accuracy}(u) \times \ln(1 + \text{comment_count}(u) + 1) \quad (6)$$

Factorul logaritmic $\ln(1 + n + 1)$ recompenseaza participarea activa (comentarii), dar cu diminuarea returnarilor — primul comentariu aduce cel mai mult, al zecelea aduce mai putin.

8 Testare si validare

Proiectul include un set comprehensiv de teste unitare si de integrare:

Clasa de test	Scop	Teste	Status
<code>CityServiceTest</code>	Servicii orase/tari	5	Pass
<code>ForecastServiceTest</code>	Prognoze, comparatii	4	Pass
<code>StatisticsServiceTest</code>	Statistici, clasamente	3	Pass
<code>UserServiceTest</code>	Voturi, comentarii	3	Pass
<code>AccuracyServiceTest</code>	Scoruri, reputatie	2	Pass
<code>SessionManagerTest</code>	Management sesiune	3	Pass
<code>DatabaseConnectionPoolTest</code>	Pool HikariCP	3	Pass
<code>DatabaseInitializerTest</code>	Migrari DB	3	Pass
<code>ClusteringServiceTest</code>	K-means, silhouette	9	Pass
<code>HmmTrainingServiceTest</code>	Forward-backward, Baum-Welch	5	Pass
<code>MLPipelineIntegrationTest</code>	Pipeline end-to-end	1	Pass
TOTAL		63	63 Pass

Tabela 4: Rezultate teste

Testul de integrare `MLPipelineIntegrationTest` valideaza intregul pipeline ML cu date reale de la Open-Meteo pentru Bucuresti: importa 365 de zile istorice, construiește vectori, ruleaza clustering, Markov, HMM si Monte Carlo, apoi verifica consistenta percentilelor ($P_{10} \leq P_{50} \leq P_{90}$) si faptul ca toate probabilitatile sunt in $[0, 1]$.

9 Bibliografie

Bibliografie

- [1] Rabiner, L. R. (1989). "A tutorial on hidden Markov models and selected applications in speech recognition." *Proceedings of the IEEE*, 77(2), 257–286. Aplicatia extinde HMM-urile pentru modelarea regimurilor meteorologice, folosind algoritmul Baum-Welch pentru estimarea parametrilor.
- [2] Norris, J. R. (1998). *Markov Chains*. Cambridge University Press. Lanturile Markov de ordin 2 implementate in `sp_build_markov_tensor` permit capturarea persistentei fenomenelor meteorologice pe 3 zile consecutive, spre deosebire de modelele de ordin 1.
- [3] Robert, C. P., & Casella, G. (2004). *Monte Carlo Statistical Methods*. Springer. Simularea cu 5000 de traectorii si extragerea percentilelor (P_{10} , P_{50} , P_{90}) urmeaza metodologia clasica a estimarii distributiilor predictive prin resampling.
- [4] Ross, T. J. (2010). *Fuzzy Logic with Engineering Applications*. Wiley. Detectoarele fuzzy (canicula, furtuna, ceata) din `sp_compute_recipe_scores` folosesc functii de apartenenta sigmoidale si operatori T-norm (**LEAST**) conform teoriei sistemelor fuzzy.

- [5] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation* — <https://www.postgresql.org/docs/16/>. Procedurile stocate PL/pgSQL, functiile de fereastra (LAG, LEAD), tipurile array si JSONB sunt documentate oficial.
- [6] Zippenfenig, P. (2023). "Open-Meteo.com Weather API." <https://open-meteo.com/>. Sursa gratuita de date meteorologice istorice si de prognoza, utilizata pentru popularea automata a tabeli `forecasts`.
- [7] Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems*. Pearson. Capitolele privind normalizarea (3NF, BCNF) si proiectarea diagramelor E/A au constituit baza teoretica a schemei bazei de date.
- [8] Oracle Corporation. *The Java Tutorials — JDBC Basics*. <https://docs.oracle.com/javase/tutorial/jdbc/>. Conectivitatea JDBC directa (fara ORM) a fost aleasa pentru a demonstra controlul explicit al transactiilor si al apelurilor de proceduri stocate.
- [9] brettwooldridge. *HikariCP*. <https://github.com/brettwooldridge/HikariCP>. Pool-ul de conexiuni HikariCP gestioneaza eficient conexiunile TCP catre PostgreSQL, reducand overhead-ul de initializare la fiecare query.
- [10] Arthur, D., & Vassilvitskii, S. (2007). "k-means++: The advantages of careful seeding." *SODA '07*. Clustering-ul regimurilor meteorologice foloseste k-means++ prin Apache Commons Math pentru initializarea centroidelor.

10 Concluzii

Prezentul proiect demonstreaza ca un sistem de gestiune a bazelor de date relationale (PostgreSQL) poate gazdui nu doar operatii CRUD, ci si algoritmi sofisticati de machine learning si simulare statistica. Prin transferarea pipeline-ului de predictie la nivelul serverului, am obtinut:

- **Coerenta tranzactionala:** actualizarile datelor si ale modelului sunt atomice;
- **Performanta:** eliminarea transferului de date masive intre server si client;
- **Auditabilitate:** fiecare parametru al modelului este istorisit in tabele relationale.

Schema bazei de date, proiectata in **BCNF**, extinde modelul clasic entitate-relatie cu tabele specializate pentru vectori, regimuri, tranzitii Markov, stari HMM si predictii Monte Carlo. Toate tabelele sunt populate automat prin scripturi de migrare si import de la API-ul Open-Meteo.

Directii viitoare:

1. Implementarea unui model HSMM (Hidden Semi-Markov Model) pentru capturarea duratei regimurilor;
2. Parallelizarea simularii Monte Carlo cu `pg_parallel`;
3. Integrarea cu modelul de limbaj pentru generarea de rapoarte meteo in limbaj natural.